

High-Impact Examples of the Optirando™ System in Action

Multimodal Intent Validation with WR Code® Scanning, the WR Graph™, and Proof of Authoritative Execution

Oscar Schreyer

Hamburg, Germany

July 2026

Abstract

Contemporary AI assistants answer the question the user asks. In operational settings this is frequently the wrong objective: users routinely ask well-formed questions that are nevertheless incorrect, because constraints already recorded in their own environment — licensing, compliance, device history, workflow state, dependencies — are not present in their working memory at the moment of asking. The Optirando™ system validates the question before answering it. A WR Code® scan anchors the interaction to a known context: a device, a machine, a document, a workflow, a dashboard, a website, a piece of software, a cash register, a room — any object or surface, physical or digital, to which a code can be attached. The smartphone camera captures on-screen error states and physical conditions directly as evidence. The WR Graph™ (Workflow-Ready Dynamic Knowledge Graph) is the system’s unifying context layer: an ordered, self-optimizing, multidimensional project knowledge graph coupled with long-term memory, carrying fine-tuned domain adapters as context-bound competence, and embedding Workflow Context Recorder automations — semantically annotated recordings that execute deterministic yet adaptive workflows. Questions are validated against this layer; a local Host AI performs all inference, hot-loading the adapter the scanned context selects; BEAP™ synchronizes field sessions to the desk as inspectable, text-pure content; and Proof of Authoritative Execution (PoAE™) ensures that consequential actions are demonstrably authorized by a human — by fingerprint or hardware key in high-assurance cases. Recorded automations run to the authorization boundary and no further. Solution paths are prepared speculatively in the background via priority-ordered tree-of-thought exploration, so validated actions are available in perceived real time. This paper introduces the components concisely and then demonstrates the composed system through high-impact examples spanning industrial maintenance, medical technology, IT operations, cybersecurity triage, regulated document workflows, software architecture, home infrastructure, onboarding, and chain-of-custody verification.

Keywords: intent validation, WR Code®, knowledge graphs, workflow recording, multimodal interaction, human-in-the-loop, cryptographic authorization, local inference, fine-tuned adapters, governed AI execution, high-assurance systems

1. Introduction

The starting point of this work was conventional intent detection for user support: given an utterance, classify what the user wants, then satisfy it. Its continuation, presented here, rests on the observation that in real operational environments the harder failure mode occurs one step earlier. A question can be clear, answerable, and correctly answered — and still be the wrong question. Real intent is rarely contained in the prompt alone: it depends on the object or screen in front of the user, the workflow

currently active, prior architectural decisions, technical and legal constraints, and the level of authority the intended action requires. The user asks about the option they happen to know, not the option their situation actually requires — and the interaction succeeds locally while failing globally: the right answer to the wrong question.

A concrete instance from this system's own development illustrates the pattern. When a local AI runtime had to be integrated into the orchestrator, the author asked about Ollama — a reasonable, well-informed question that received correct answers. Only later did it emerge that two constraints fundamental to the project, and recorded in its decision history, had never entered the question: licensing posture and system latency. Evaluated against those constraints, llama.cpp was the better-suited runtime, and the eventual migration cost substantial engineering effort that a validated question would have avoided. A capable system holding the project's context would have asked the follow-up a knowledgeable colleague asks: is there a runtime that fits your licensing and latency requirements better? It would have found out what the user actually needed.

Correcting questions requires context the user does not hold in the moment but their environment already contains — in structured, queryable form. The Optirando™ system assembles this capability from a small set of components, each individually modest, whose composition produces a pipeline: anchor the context by scan, capture the evidence visually, validate the question against the WR Graph™ — the multidimensional, self-optimizing knowledge graph coupled with system memory — prepare solution paths speculatively in the background, and gate execution behind a cryptographically provable human decision. Where a validated path corresponds to a recorded workflow, preparation extends to instantiating that workflow up to — but never across — the authorization boundary. The result is a shift from prompt answering — and equally from blind automation — to context-bounded intent resolution and governed execution. Section 2 introduces the components; Section 3 demonstrates the composed system through high-impact examples.

2. System Components

2.1 WR Code® — The Universal Context Anchor

A WR Code® is a scan-based context anchor, and its defining property is that it can be placed anywhere. On physical objects: a printer, a machine tool, an infusion pump, a server rack, a cash register, a document container — or the office coffee machine. Equally on digital and organizational surfaces: a website, a software dashboard, a repository, a ticket, a workflow stage, a process page, a room, a workstation. Scanning it resolves the context deterministically — no vision-based guessing, no recognition uncertainty compounding with intent uncertainty. Unlike an opaque automation trigger, a WR Code® carries no hidden semantics: it is a transparent, inspectable entry point that tells the system “this request belongs to this known object, process, node and publisher identity.” The scan collapses the interpretation space from “anything the user might mean” to “this context, its state, its history,” and simultaneously selects the relevant subgraph of the WR Graph™ — and with it the matching domain adapter and any recorded automations bound to that context.

2.2 Multimodal Capture — The Camera as Evidence Channel

After the scan, the user adds field input: a photograph of an error message on a screen, a defective component, a screenshot, a spoken or typed question. This eliminates the weakest link of classical

support interactions — the user’s transcription and paraphrase — and carries exact codes, device state, and incidental detail the user would not think to mention. Many real-world problems are simply not expressible in text alone; the smartphone becomes the bridge between the situation and the system. The captured image plays a dual role: it is an intent signal at the start of the pipeline and evidence at its end, where it is bound into the authorization record.

2.3 WR Graph™ — The Unifying Context Layer

The WR Graph™ (Workflow-Ready Dynamic Knowledge Graph) is the system’s unifying context layer, and it unites three capabilities in one structure: an ordered, multidimensional, self-optimizing project knowledge graph coupled with long-term interaction memory; fine-tuned, LoRA-class domain adapters carried in the graph as context-bound competence; and Workflow Context Recorder automations — semantically annotated workflow recordings embedded as executable, inspectable paths. Where the WR Code® narrows context spatially, the WR Graph™ supplies it in depth — and, through its recorded automations, supplies not only what is known about a context but what can be done in it.

This is what distinguishes the WR Graph™ from a knowledge graph in the conventional sense: it holds a domain not in one representation but in four — declarative knowledge in the graph structure, episodic knowledge in memory, parametric knowledge in adapter weights and embeddings, and procedural knowledge in recorded workflows. A single scan selects all four at once: what is known about a context, what has happened in it, what competence it demands, and what can be done in it. The constituents are described in turn.

2.3.1 Graph and Memory — Contextual Understanding

The knowledge graph is a structured wiki-like representation of systems, objects, users, roles, policies, vendors, decisions, workflows, risks, and their relations — ordered, multidimensional, and self-optimizing: usage, corrections, and authorized decisions continuously refine its structure and priorities. It supplies the depth the scan cannot: which dependencies matter, which constraints apply (licensing, compliance, warranty, safety, retention), which previous decisions govern the situation, which action paths are available and authorized. Memory adds continuity: recurring issues, prior incidents, known false positives, preferences, and the rationale behind past choices. Question validation is performed against graph and memory, not against general world knowledge — because they hold exactly the knowledge the user is most likely missing at the moment of asking.

2.3.2 Fine-Tuned Adapters — Domain Expertise on Demand

The graph carries specialized, fine-tuned lightweight adapters (LoRA-class) as first-class context: provisioned through the graph and hot-loaded by the host at inference time. The scanned WR Code® determines which adapter applies: a maintenance adapter tuned on fault codes and service procedures, a medical-device adapter tuned on device records and regulatory vocabulary, a cybersecurity adapter for alert classification, a procurement adapter for order preparation, a compliance adapter for regulated workflows, a DevOps adapter for runbooks and topology reasoning. Adapter selection is thus itself context-driven — the same scan that narrows the knowledge also sharpens the model — while the base model, adapters, and their weights remain entirely local.

2.3.3 Workflow Context Recorder — Deterministic yet Adaptive Execution Paths

Embedded in the graph are Workflow Context Recorder automations: semantically annotated recordings whose steps are stored as intent, not as positional replay. Execution is deterministic where it must be — tool invocations and consequential actions are cryptographically bound, fixed, and inspectable — and adaptive where it may be: the semantic step sequence resolves against the current interface state, so UI changes do not immediately break an automation. Frequently no adaptation is required at all, because the WR Code® publisher controls the surface itself — the software, the site, the dashboard — and recording and interface evolve together under one governance.

This places the Recorder precisely between two failing extremes. Classical RPA records positionally and breaks with every interface change; unconstrained agents are robust to change but neither reproducible nor auditable. A recorded automation is robust on the way and rigid at the point of consequence. And it respects the system's central boundary: recorded automations run to the authorization boundary and no further — any step whose consequence reaches the applicable PoAE™ tier is staged, not executed.

2.4 Host AI — Local Inference

All inference runs on a host machine within the user's trust domain — in the simplest deployment, a PC at home or on premises that is online and paired with the smartphone. The phone captures; the host reasons. The Host AI can reach resources no generic cloud chatbot can: project files, local databases, configuration notes, monitoring data, logs, inventory and procurement systems, code repositories, locally deployed models. Multimodal inputs, the WR Graph™, memory, adapters, recorded automations, and inference never leave the user's control. This is a precondition, not a preference: in the regulated environments of Section 3, cloud multimodal assistants are excluded by data-protection and compliance policy, and a locally inferring host is the only admissible substrate.

2.5 BEAP™ — Secure Text-Pure Transport

The Bidirectional Email Automation Protocol (BEAP™) transports session state securely between endpoints — here, from the field session on the smartphone to the desk environment on the host. BEAP™ enforces text purity: content is constrained to a positively constructed allowlist, structurally excluding payload-borne attack classes; a capsule can be read, logged, validated, and inspected rather than trusted as opaque binary state. The scan reference, the structured description of captured evidence, the validated intent, and the prepared solution paths — including a proposed action awaiting confirmation — synchronize to the desk as text, so field-to-desk continuity does not widen the attack surface.

2.6 PoAE™ — Proof of Authoritative Execution

The system validates, corrects, prepares, and stages — it never executes authoritatively on its own. The principle is: the AI prepares, the human authorizes, the system records proof of that authorization. PoAE™ makes the human act cryptographically demonstrable: in critical, high-assurance cases the confirmation is bound to the person via fingerprint or hardware security key. Two properties distinguish this from a confirmation dialog. First, evidential binding: the authorization is linked to the WR Code® scan, the photographed state, the corrected question, and the approved action — an auditor later sees person, time, action, and evidence as one cryptographic record, not “the system suggested it.” Second,

risk-adaptive gradation: authorization strength is derived from the same graph context that validated the question, scaling from a simple tap for a consumables order, through biometric confirmation for financially or legally relevant acts, to dual hardware-key signatures implementing a cryptographic four-eyes principle on critical infrastructure. This is the difference between automation and governed execution.

2.7 Background Preparation — Priority-Ordered Tree of Thought

While the user is still capturing and composing, the system explores candidate solution paths via tree-of-thought reasoning, ranked by priority, cost, speed, safety, and risk: the most likely and most consequential branches first. Preparation means everything short of execution — drafting the order, staging the service ticket, sequencing the runbook, assembling the containment checklist, pre-filling the compliance record, or instantiating a recorded automation up to its authorization boundary. By the time the user has finished asking, the validated action is typically ready: the interaction is experienced in real time although substantive multi-step work has occurred, and execution remains gated behind PoAE™.

3. High-Impact Examples

The examples are ordered from the everyday to the highly regulated, and deliberately alternate between physical and digital anchors — a machine display and a compliance workflow, a server rack and a software dashboard. Each exercises the full pipeline — anchor, evidence, adapter-sharpened validation, background preparation, authorized execution — and each turns on the system knowing something the user, in the moment, does not.

3.1 Office Consumables: The Real-Time Baseline

A user scans the WR Code® on an office printer, photographs the message on its display, and asks: “What should I do?” The graph links the printer to its model, location, consumables, vendor, and support contract; the adapter distinguishes empty toner from a paper jam, a maintenance warning, or a hardware fault. If the correct cartridge is already in storage, the answer is where to find it; if not, the reorder is prepared in the background while the user is still typing — a recorded ordering workflow, instantiated with the correct cartridge and vendor from the graph, run to the point of release — and execution is a single tap: the lowest PoAE™ tier, proportionate to the stakes. The example is deliberately mundane: it demonstrates the perceived-real-time property, the field-to-execution continuity, and the authorization floor before the same pipeline carries decisions of entirely different weight.

3.2 Industrial Maintenance: Economic Question Correction

A technician scans a production machine, photographs the fault code on its HMI panel, and asks how to reset it. The question is reasonable, answerable — and, according to the graph, wrong. The maintenance history shows the same fault recurring for the third time in four weeks: the signature of a degrading component, not a transient error. A reset would clear the symptom and defer the failure to a worse moment. Validation surfaces exactly this; the maintenance adapter recognizes the fault pattern for this machine class; and both paths arrive prepared — the reset procedure, and a service order with the part reserved, the approved vendor and cost center identified from the graph, and warranty pre-checked. Because the replacement part is a financially relevant order, PoAE™ requires the technician or an authorized manager to confirm it — the record shows the order was not merely AI-generated but

authoritatively confirmed. The authorization, bound to scan and photograph, enters the machine's history and improves the next validation. The avoided unplanned line stop typically exceeds the component cost by orders of magnitude.

3.3 Medical Technology: Compliance Correction Under Regulation

A nurse scans an infusion pump, photographs error E-42, and asks how to clear it. The device record in the graph shows an open field safety notice for this serial number: the revision is under recall. The correct response is not the requested procedure but an escalation — take the device out of service; replacement device X is available on a nearby ward. The user's question was not suboptimal but superseded, by a constraint no individual could be expected to carry in memory. Decommissioning a medical device is a documentation-mandatory act under the EU Medical Device Regulation, and the biometric PoAE™ authorization — person, time, action, and photographed evidence, cryptographically linked — is precisely the artifact a subsequent audit requires. Equally decisive: photographs of clinical equipment cannot be sent to a cloud service under any defensible hospital policy. Local Host AI with a locally loaded medical-device adapter is the admission ticket; cloud assistants are structurally excluded from this scenario.

3.4 IT Operations: Cryptographic Four-Eyes on Critical Infrastructure

A data-center operator scans a rack, photographs a switch's status LEDs, and asks whether the switch can be restarted. The graph holds the live topology and knows what the operator does not: a critical replication job is running across that switch. A restart would be answered correctly and executed disastrously. Validation reorders the plan — failover first, then restart — and the failover runbook, held in the graph as a recorded automation whose steps resolve semantically against the live operations console, is sequenced in the background. The session, captured in the rack aisle, synchronizes over BEAP™ to the operations desk; because the action touches critical infrastructure, the derived PoAE™ tier demands two independent hardware-key signatures before the run begins. Classical change management implements the four-eyes principle procedurally, through ticket approvals detached from the physical situation; this architecture implements it cryptographically, bound to the specific rack, evidence, and runbook.

3.5 Cybersecurity Triage: From Isolated Warning to Operational Event

An administrator scans the WR Code® on a security dashboard or admin panel — a digital anchor — photographs or screenshots the visible warning, and asks: “Is this dangerous?” A generic assistant would describe the warning. The context-bounded system resolves its operational meaning: the graph shows which services run on the affected system, which data categories they process, which users and dependencies are touched; memory shows whether this alert matches a known false-positive pattern or a prior incident; the cybersecurity adapter classifies the alert against the organization's incident-response process. The prepared paths differ accordingly — a documented dismissal for the known false positive, or, for the genuine case, an incident report, an evidence-collection sequence, a containment checklist, and a pre-drafted escalation message, ranked and staged in the background. Containment actions that restrict access or alter infrastructure are gated behind PoAE™ — in incident response, precisely the moment when hasty, unauditable action is most tempting and most dangerous. The failure mode this prevents is treating a security warning as an isolated technical message instead of a context-dependent operational event.

3.6 Regulated Documents and Workflows: The WR Code® as Governance Anchor

A WR Code® need not sit on hardware at all. Attached to a regulated workflow — a customer contract, an invoice approval, an HR document, a compliance process stage — it binds assistance to a process and its permission context. The user asks: “Can I send this?” The graph links the document to its workflow stage, responsible roles, data category, retention rules, and approval requirements; the compliance adapter checks what the stage demands. Validation may find that an approval is missing, that the document contains personal data requiring a different channel, or that the user’s role does not carry the authority to release it — answers to a question the user did not ask but needed answered first. Low-risk actions are simply prepared; externally binding or regulated ones synchronize as a BEAP™ capsule and require PoAE™ confirmation, leaving an auditable record of who released what, when, and on what evidence. The WR Code® here is a governance anchor: it turns “AI helped with a document” into “AI assistance bound to a specific process, permission context, and execution path.”

3.7 Software Architecture: The Ollama Case, Revisited

The introductory case becomes an example of the running system — anchored digitally, via a WR Code® on the orchestrator’s repository or dashboard. A developer asks how to integrate Ollama. The WR Graph™ holds the project’s recorded constraints — licensing sensitivity for redistribution, latency as a first-order requirement — and validation flags the mismatch: the corrected question is “which local inference runtime best satisfies this project’s licensing and latency constraints,” which leads directly to llama.cpp (with vLLM as the server-scale alternative). The same mechanism generalizes to everyday DevOps: a deployment error photographed from a dashboard may name a symptom while the graph identifies the deeper cause — an expired certificate, a fragile dependency, a missing abstraction layer — turning error explanation into architectural decision support. The developer accepts the correction, and under PoAE™ that acceptance is recorded as an authorized architectural decision with its rationale. The graph thereby accumulates not merely facts but responsible decisions — every correction it performs enriches the context for the next one.

3.8 Home Office and Local Infrastructure: Private Context-Aware Assistance

The pattern scales down to a single household or home office. A user scans the WR Code® on a router, NAS, local server, 3D printer — or on a self-hosted service’s status page — photographs the error, and asks: “Why is this not working?” The Host AI on the user’s own PC holds what no remote assistant should be given: configuration notes, container states, backup logs, certificate inventory, DNS setup, the graph of how router, domains, storage, and reverse proxy depend on one another. Reasoning within that actual environment, the system routinely finds that the visible error is not the problem — the real cause is an expired certificate, a failed container, or a DNS mismatch two dependencies away. The impact here is twofold: troubleshooting quality, because generic advice is replaced by environment-specific diagnosis; and privacy, because the entire diagnostic context remains on the user’s own hardware. It is the same architecture that serves the hospital, at domestic scale.

3.9 Onboarding and Guided Execution: From Documentation to Next Action

A new employee scans the WR Code® next to an internal tool, a process step, a workstation — or on a retail cash register — and asks: “What do I need to do here?” A generic assistant cannot know the local process; the graph does: the workflow, its required inputs, the responsible roles, the escalation paths.

Memory adds what manuals omit — the common mistakes, the recent process changes, the employee’s own onboarding progress. Instead of a manual, the system returns the next relevant action: which field to check, which approval is required, which safety step must precede, whom to contact, which error to avoid. Where the action is sensitive, preparation and execution separate as everywhere else: the new cashier processing a refund above the threshold gets the transaction fully prepared, and the manager’s PoAE™ confirmation — not the new employee’s uncertainty — releases it. Static documentation becomes operational guidance, with authority boundaries intact from day one.

3.10 Chain-of-Custody: Verification as a First-Class Question

Closest to the architecture’s origins in high-assurance communication: a field worker — journalist, NGO staff member, auditor — scans the WR Code® on a document container or item of equipment whose provenance matters, and photographs its physical state, including seals and any suspected tampering marks. The question answered is not “what is this” but “does the observed state match the last attested state in the provenance chain,” which the WR Graph™ holds. Discrepancies are surfaced; the worker’s attestation is then executed under PoAE™, biometrically bound to scan and images, extending the custody chain with an authorization record rather than a log entry. Provenance chain and authorization chain become one structure — every link simultaneously a fact and a proof of who vouched for it.

4. Discussion: The Common Pattern

Across ten scenarios — printer and production line, hospital ward and data center, compliance workflow and cash register, home server and custody chain — the pattern is invariant. The user starts with a simple question; the system does not treat it as complete. The WR Code® reduces ambiguity by anchoring the request to a known context, physical or digital. Multimodal capture fixes the current state as evidence. The WR Graph™ supplies the constraints the user is missing; its adapters sharpen the reasoning for the domain at hand; its recorded automations supply execution paths that are semantically robust on the way and cryptographically fixed at the point of action. Validation decides whether to answer, correct, or escalate the question. Background preparation makes the validated action available in perceived real time. And PoAE™ keeps the boundary of authority exact: the system prepares without executing, the human executes with proof. Risk is graded, not binary — explanation and checklist at the low end, drafted tickets and orders in the middle, and at the high end purchases, infrastructure changes, security containment, and regulated communication, each requiring authorization proportionate to its consequence. The highest impact concentrates where two conditions intersect: a wrong question carries safety, compliance, or economic cost, and cloud-based assistance is excluded by policy. At that intersection, a locally inferring, graph-grounded, cryptographically governed assistant is not one option among several — it is the only admissible one.

5. Conclusion

Intent detection asked what the user means. Intent validation asks whether what the user means is what the user needs — a question only answerable by a system that holds the user’s context in structured, queryable form and keeps it within the user’s trust domain. The WR Code® makes any object, screen, workflow, or place — from an industrial press to a compliance process to the office coffee machine — a deterministic entry point into that context. The WR Graph™ unifies what that context knows, what

competence it demands, and what it can execute: graph and memory, hot-loaded domain adapters, and semantically recorded workflows that remain adaptive on the way and deterministic at the point of consequence. Composed with multimodal capture, local inference, BEAP™ transport, and PoAE™ authorization, the components produce interactions in which the system prepares in the background what a knowledgeable colleague would have suggested, and the user executes, in perceived real time, what they actually needed rather than what they happened to ask — with cryptographic proof, afterwards, that a human made the call. The central shift is simple: AI should not only answer what the user asks; it should help determine what the user actually needs — and when action matters, it should prepare execution without ever replacing human authority.